

Sprint 9 — Decorator Pattern

Class 1 — Why Decorator Exists

Learning Objectives

By the end of this class, we should be able to:

- Explain why Decorator Pattern exists.
- Recognize the inheritance explosion problem.
- Understand **Component**, **Concrete Component**, **Decorator**, and **Concrete Decorator**.
- Add behavior without modifying an existing class.
- Combine multiple behaviors dynamically.
- Understand why Decorator favors **composition over inheritance**.
- Connect Decorator to SOLID principles.
- Recognize when Decorator should **not** be used.

Lesson 1 — Start With a Real Problem

Suppose we are building a notification system.

Initially, the requirement is simple:

Send notification by email.

We create an interface:

```
public interface NotificationSender {  
  
    void send(String message);  
}
```

Implementation:

```
public class EmailNotificationSender  
    implements NotificationSender {
```

```
@Override
public void send(String message) {
    System.out.println(
        "Send Email: " + message
    );
}
}
```

Client:

```
public class NotificationDemo {

    public static void main(String[] args) {

        NotificationSender sender =
            new EmailNotificationSender();

        sender.send(
            "Your booking is confirmed."
        );
    }
}
```

Everything looks good.

Lesson 2 — Requirements Start Growing

A few weeks later, business says:

We need to log every notification.

A developer may create:

```
public class LoggedEmailNotificationSender
    extends EmailNotificationSender {
```

```

@Override
public void send(String message) {

    System.out.println(
        "LOG: Sending notification"
    );

    super.send(message);
}
}

```

It works.

Then another requirement arrives:

Some notifications must be encrypted.

We create:

```

public class EncryptedEmailNotificationSender
    extends EmailNotificationSender {

    @Override
    public void send(String message) {

        String encrypted =
            encrypt(message);

        super.send(encrypted);
    }

    private String encrypt(String message) {
        return "ENCRYPTED[" + message + "]";
    }
}

```

Still manageable.

Then business says:

We need logging AND encryption.

Now what?

```
public class LoggedEncryptedEmailNotificationSender
    extends EmailNotificationSender {
}
```

Later:

Add retry.

Now we may need:

RetryEmailNotificationSender

RetryLoggedEmailNotificationSender

RetryEncryptedEmailNotificationSender

RetryLoggedEncryptedEmailNotificationSender

Then:

Add metrics.

Now combinations explode.

Lesson 3 — The Inheritance Explosion Problem

Suppose we have four optional behaviors:

Logging

Encryption

Retry
Metrics

Possible combinations include:

Email

LoggedEmail

EncryptedEmail

RetryEmail

MetricsEmail

LoggedEncryptedEmail

LoggedRetryEmail

LoggedMetricsEmail

EncryptedRetryEmail

EncryptedMetricsEmail

RetryMetricsEmail

LoggedEncryptedRetryEmail

LoggedEncryptedMetricsEmail

LoggedRetryMetricsEmail

EncryptedRetryMetricsEmail

LoggedEncryptedRetryMetricsEmail

This is the problem.

The behavior is optional and composable, but inheritance makes the combinations static.

Question

What happens when we add:

Audit

Rate Limiting

Tracing

Caching

Do we keep creating subclasses?

No.

We need another design.

Lesson 4 — Why Decorator Exists

Decorator exists because sometimes we want to:

Add responsibilities to an object dynamically without modifying the original class.

Instead of:

Email

—— LoggedEmail

—— EncryptedEmail

—— RetryEmail

—— LoggedEncryptedEmail

—— RetryLoggedEncryptedEmail

we build objects through composition:

LoggingDecorator
|
RetryDecorator
|
EncryptionDecorator
|
EmailNotificationSender

Every object implements the same interface: NotificationSender

Therefore, the client still sees: `sender.send(message);`

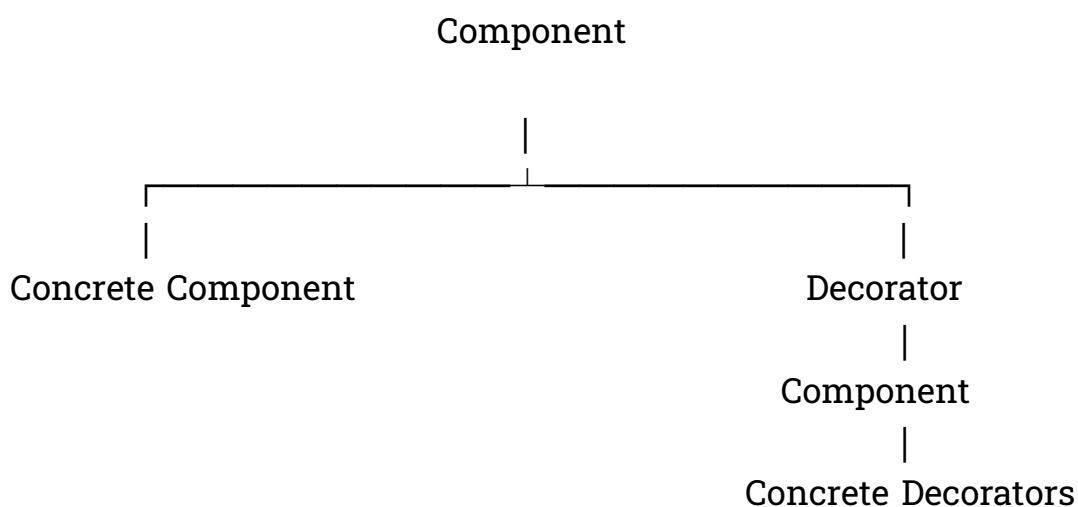
Key Definition

Decorator Pattern attaches additional responsibilities to an object dynamically while keeping the same interface.

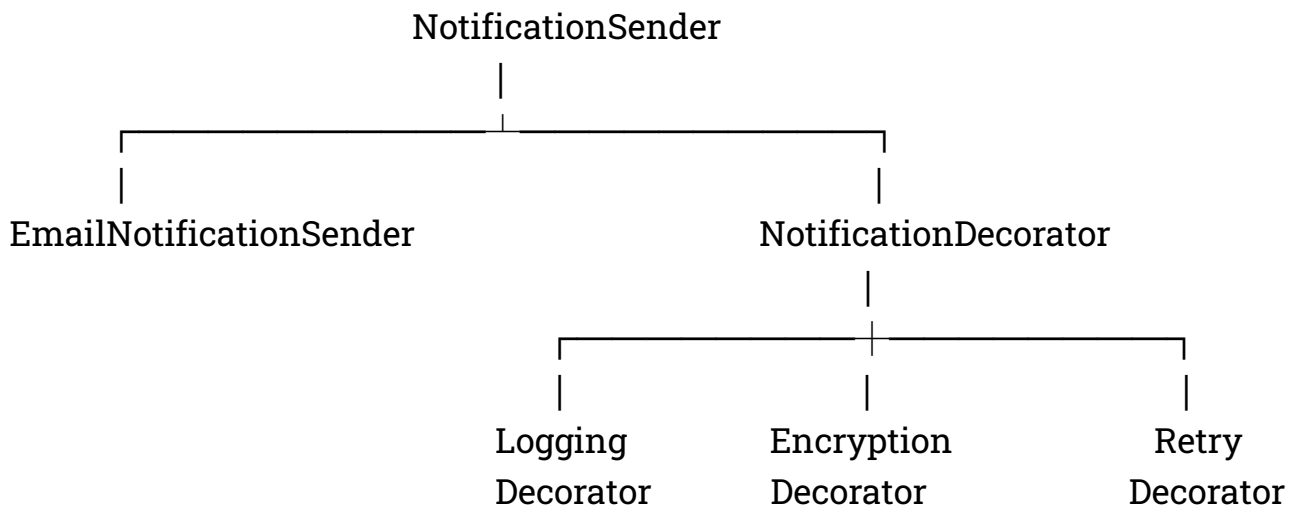
Decorator wraps an object and adds behavior around it.

Lesson 5 — Decorator Structure

The classic structure is:



In our example:



Lesson 6 — Component Interface

First, create the common interface:

```
public interface NotificationSender {  
  
    void send(String message);  
}
```

This is called the **Component**.

Both:

- Original implementation
- Decorators

implement this interface.

This is one of the most important parts of Decorator Pattern.

Lesson 7 — Concrete Component

The real implementation:

```
public class EmailNotificationSender
    implements NotificationSender {

    @Override
    public void send(String message) {

        System.out.println(
            "EMAIL: " + message
        );
    }
}
```

This is the Concrete Component

It performs the original behavior.

Lesson 8 — Base Decorator

Now create the base Decorator:

```
public abstract class NotificationDecorator
    implements NotificationSender {

    protected final NotificationSender delegate;

    protected NotificationDecorator(
        NotificationSender delegate
    ) {
        this.delegate = delegate;
    }
}
```

Notice something important:

NotificationDecorator implements NotificationSender
and contains: NotificationSender delegate;

This is the heart of Decorator Pattern.

Important Structure

Decorator

IS-A NotificationSender

Decorator

HAS-A NotificationSender

That means:

IS-A

+

HAS-A

at the same time.

This allows decorators to be nested.

Lesson 9 — Logging Decorator

```
public class LoggingNotificationDecorator
    extends NotificationDecorator {

    public LoggingNotificationDecorator(
        NotificationSender delegate
    ) {
        super(delegate);
    }
}
```

```

    }

    @Override
    public void send(String message) {

        System.out.println(
            "[LOG] Sending notification"
        );

        delegate.send(message);

        System.out.println(
            "[LOG] Notification sent"
        );
    }
}

```

Usage:

```

NotificationSender sender =
    new LoggingNotificationDecorator(
        new EmailNotificationSender()
    );

sender.send(
    "Booking confirmed"
);

```

Output:

```

[LOG] Sending notification
EMAIL: Booking confirmed
[LOG] Notification sent

```

We added logging without modifying:

EmailNotificationSender

Lesson 10 — Encryption Decorator

Now add another behavior.

```
public class EncryptionNotificationDecorator
    extends NotificationDecorator {

    public EncryptionNotificationDecorator(
        NotificationSender delegate
    ) {
        super(delegate);
    }

    @Override
    public void send(String message) {

        String encryptedMessage =
            encrypt(message);

        delegate.send(
            encryptedMessage
        );
    }

    private String encrypt(
        String message
    ) {
        return "ENCRYPTED[" + message + "]";
    }
}
```

Usage:

```
NotificationSender sender =  
    new EncryptionNotificationDecorator(  
        new EmailNotificationSender()  
    );
```

Output:

EMAIL: ENCRYPTED[Booking confirmed]

Again:

EmailNotificationSender was not modified.

Lesson 11 — Combining Decorators

This is where Decorator becomes powerful.

We can combine them:

```
NotificationSender sender =  
    new LoggingNotificationDecorator(  
        new EncryptionNotificationDecorator(  
            new EmailNotificationSender()  
        )  
    );
```

Structure:

```
LoggingDecorator  
    |  
EncryptionDecorator  
    |  
EmailNotificationSender
```

Call:

```
sender.send(  
    "Your booking is confirmed"  
);
```

Execution:

Logging before

↓

Encryption

↓

Email

↓

Logging after

Output:

[LOG] Sending notification

EMAIL: ENCRYPTED[Your booking is confirmed]

[LOG] Notification sent

Lesson 12 — Add Metrics Without Changing Anything

Create another decorator:

```
public class MetricsNotificationDecorator  
    extends NotificationDecorator {  
  
    public MetricsNotificationDecorator(  
        NotificationSender delegate  
    ) {  
        super(delegate);  
    }  
}
```

```

@Override
public void send(String message) {

    long start =
        System.nanoTime();

    try {

        delegate.send(message);

    } finally {

        long duration =
            System.nanoTime() - start;

        System.out.println(
            "[METRICS] duration="
                + duration
                + "ns"
        );
    }
}

```

Now:

```

NotificationSender sender =
    new MetricsNotificationDecorator(
        new LoggingNotificationDecorator(
            new EncryptionNotificationDecorator(
                new EmailNotificationSender()
            )
        )
    );

```

No existing implementation changed.

Lesson 13 — Runtime Composition

This is a major advantage over inheritance.

Suppose configuration determines behavior:

```
NotificationSender sender =  
    new EmailNotificationSender();  
  
if (config.encryptionEnabled()) {  
    sender =  
        new EncryptionNotificationDecorator(  
            sender  
        );  
}  
  
if (config.loggingEnabled()) {  
    sender =  
        new LoggingNotificationDecorator(  
            sender  
        );  
}  
  
if (config.metricsEnabled()) {  
    sender =  
        new MetricsNotificationDecorator(  
            sender  
        );  
}
```

The behavior can now be assembled at runtime.

For example:

Production

Metrics

↓
Logging
↓
Encryption
↓
Email

But development might use:

Logging
↓
Email

Same classes.

Different composition.

Lesson 14 — Decorator Order Matters

This is extremely important.

Compare:

```
new LoggingNotificationDecorator(  
    new EncryptionNotificationDecorator(  
        email  
    )  
);
```

with:

```
new EncryptionNotificationDecorator(  
    new LoggingNotificationDecorator(  
        email  
    )  
);
```

They are not necessarily equivalent.

Consider:

Logging



Encryption



Email

versus:

Encryption



Logging



Email

If logging records the message, one design might log:

Booking confirmed

while another could log:

ENCRYPTED[Booking confirmed]

The order of decorators can affect behavior.

Lesson 15 — Before vs After Behavior

A decorator can add behavior:

Before

```
@Override
public void send(String message) {

    logBefore();

    delegate.send(message);
}
```

After

```
@Override
public void send(String message) {

    delegate.send(message);

    logAfter();
}
```

Before and After

```
@Override
public void send(String message) {

    logBefore();

    delegate.send(message);

    logAfter();
}
```

Transform Input

```
@Override
public void send(String message) {
```

```
String encrypted =  
    encrypt(message);  
  
    delegate.send(encrypted);  
}
```

Transform Output

If the method returns a result:

```
@Override  
public PaymentResult pay(  
    PaymentRequest request  
) {  
  
    PaymentResult result =  
        delegate.pay(request);  
  
    return enrich(result);  
}
```

This makes Decorator very flexible.

Lesson 16 — Real Enterprise Example: Payment

Now move from the simple notification example to something we will see in enterprise applications.

We already have:

```
public interface PaymentGateway {  
  
    PaymentResult pay(  

```

```
        PaymentRequest request
    );
}
```

Real implementation:

```
public class AbaPaymentAdapter
    implements PaymentGateway {

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        System.out.println(
            "Call ABA Payment API"
        );

        return new PaymentResult(
            "TXN-1001",
            true
        );
    }
}
```

Remember:

AbaPaymentAdapter is using **Adapter Pattern**.

Now we want:

Logging
Metrics
Retry

Do not modify:

AbaPaymentAdapter

Decorate it.

Logging Payment Decorator

```
public class LoggingPaymentDecorator
    implements PaymentGateway {

    private final PaymentGateway delegate;

    public LoggingPaymentDecorator(
        PaymentGateway delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        System.out.println(
            "[LOG] Payment started"
        );

        PaymentResult result =
            delegate.pay(request);

        System.out.println(
            "[LOG] Payment finished: "
                + result.transactionId()
        );

        return result;
    }
}
```

```
}
```

Metrics Payment Decorator

```
public class MetricsPaymentDecorator
    implements PaymentGateway {

    private final PaymentGateway delegate;

    public MetricsPaymentDecorator(
        PaymentGateway delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        long start =
            System.nanoTime();

        try {

            return delegate.pay(request);

        } finally {

            long duration =
                System.nanoTime() - start;

            System.out.println(
                "[METRICS] payment.duration="
                + duration
            );
        }
    }
}
```

```

    );
  }
}

```

Final Composition

```

PaymentGateway gateway =
    new MetricsPaymentDecorator(
        new LoggingPaymentDecorator(
            new AbaPaymentAdapter()
        )
    );

```

Architecture:

```

Client
|
MetricsPaymentDecorator
|
LoggingPaymentDecorator
|
AbaPaymentAdapter
|
ABA API

```

This example is excellent because we can see two patterns working together:

```

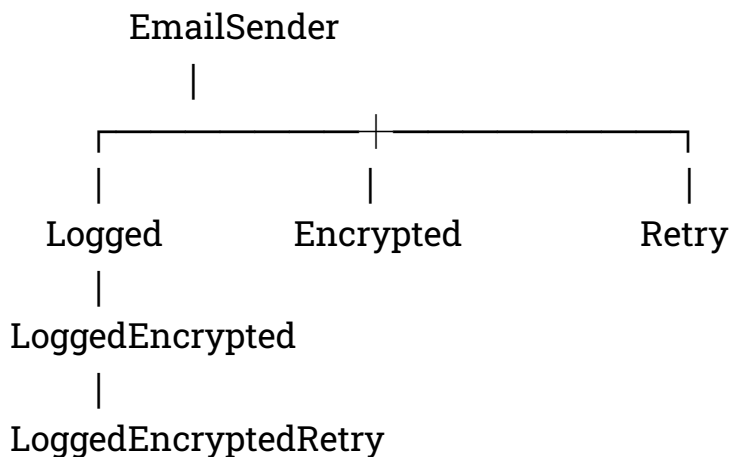
Decorator
+
Adapter

```

Lesson 17 — Composition Over Inheritance

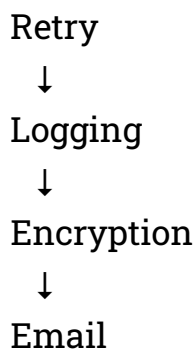
This is one of the most important lessons behind Decorator.

Inheritance:



The hierarchy grows.

Decorator:



Behavior is assembled through objects.

Inheritance chooses behavior when we design the class. Decorator lets us compose behavior when we build the object.

Lesson 18 — SOLID Connection

Open/Closed Principle

Suppose this class already works:

```
public class EmailNotificationSender  
    implements NotificationSender {  
}
```

Business asks for metrics.

We don't modify it.

We add:

```
public class MetricsNotificationDecorator  
    extends NotificationDecorator {  
}
```

The system is:

Open for extension, closed for modification.

This is one of the clearest examples of OCP.

Single Responsibility Principle

Without Decorator:

```
public class EmailNotificationSender {  
  
    public void send(String message) {  
  
        validate();  
  
        encrypt();  
    }  
}
```

```
        log();

        startMetrics();

        retry();

        sendEmail();

        audit();

        stopMetrics();
    }
}
```

This class has many reasons to change.

With Decorator:

EmailSender → Email responsibility

EncryptionDecorator → Encryption responsibility

LoggingDecorator → Logging responsibility

MetricsDecorator → Metrics responsibility

RetryDecorator → Retry responsibility

Responsibilities are separated.

Dependency Inversion Principle

Clients depend on:

NotificationSender

not:

EmailNotificationSender

or:

LoggingNotificationDecorator

Example:

```
public class NotificationService {  
  
    private final NotificationSender sender;  
  
    public NotificationService(  
        NotificationSender sender  
    ) {  
        this.sender = sender;  
    }  
  
    public void notify(  
        String message  
    ) {  
        sender.send(message);  
    }  
}
```

The client doesn't care whether it receives:

Email

or:

Metrics

↓

Logging

↓

Encryption

↓

Email

Both are: NotificationSender

Lesson 19 — Decorator vs Simple Wrapper

We may ask:

Is every wrapper a Decorator?

No.

A Decorator normally preserves the same abstraction.

Example:

```
public class LoggingPaymentDecorator
    implements PaymentGateway {

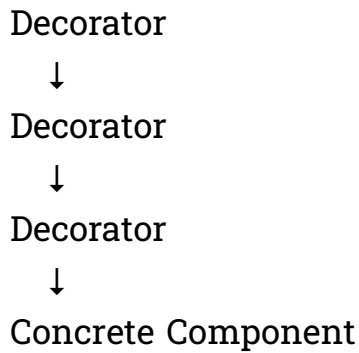
    private final PaymentGateway delegate;
}
```

Notice:

implements PaymentGateway

HAS PaymentGateway

This allows:



A random wrapper does not necessarily have this structure.

Lesson 20 — Common Mistakes

Mistake 1 — Decorator Does Not Implement the Same Interface

Bad:

```
public class LoggingService {  
  
    private final NotificationSender sender;  
}
```

This may be a wrapper, but clients cannot transparently substitute it for:

NotificationSender

Better:

```
public class LoggingNotificationDecorator  
    implements NotificationSender {  
  
}
```

Mistake 2 — Modifying the Original Component Anyway

Bad:

```
public class EmailNotificationSender {  
  
    boolean loggingEnabled;  
    boolean metricsEnabled;  
    boolean encryptionEnabled;  
    boolean retryEnabled;  
}
```

Soon:

```
if (loggingEnabled) {  
}  
  
if (metricsEnabled) {  
}  
  
if (encryptionEnabled) {  
}  
  
if (retryEnabled) {  
}
```

You are moving optional responsibilities into the core class.

Decorator exists specifically to avoid this.

Mistake 3 — Creating One Giant Decorator

Bad:

```
public class EnterpriseNotificationDecorator {  
  
    void log() {  
    }  
  
    void encrypt() {  
    }  
  
    void retry() {  
    }  
  
    void audit() {  
    }  
  
    void collectMetrics() {  
    }  
  
    void rateLimit() {  
    }  
}
```

You have recreated the original problem.

Prefer:

LoggingDecorator
EncryptionDecorator
RetryDecorator
AuditDecorator
MetricsDecorator
RateLimitDecorator

Each one has a focused responsibility.

Mistake 4 — Ignoring Decorator Order

This:

Retry



Metrics



Payment

may behave differently from:

Metrics



Retry



Payment

Why?

Suppose payment fails twice and succeeds on the third attempt.

If Metrics wraps Retry:

Metrics



Retry



Payment

Metrics may measure the complete operation including all retries.

But:

Retry



Metrics



Payment

may measure each attempt separately.

This is a very good senior-level interview discussion.

Mistake 5 — Using Decorator for Completely Different Interfaces

Suppose we have:

PaymentGateway

but an external SDK exposes:

AbaSdk

Decorator is not the first solution.

We need:

PaymentGateway



AbaPaymentAdapter



AbaSdk

Then we can decorate the Adapter:

MetricsDecorator



RetryDecorator

↓
LoggingDecorator
↓
AbaPaymentAdapter
↓
ABA SDK

Lesson 21 — When Should We Use Decorator?

Use Decorator when:

- You need optional behaviors.
- Behaviors need different combinations.
- Behavior should be added without modifying the original implementation.
- Subclass combinations are exploding.
- You want runtime composition.
- Several cross-cutting responsibilities share the same abstraction.

Examples:

Logging
Metrics
Retry
Caching
Compression
Encryption
Auditing
Tracing
Authorization

When NOT to Use Decorator

Do not automatically use Decorator for every cross-cutting concern.

Avoid it when:

- There is only one simple implementation.
- Behavior will never be combined.
- The wrapper chain becomes harder to understand than the original code.
- Framework features already solve the problem more clearly.
- The behavior belongs naturally inside the domain object.
- You actually need Adapter, Proxy, Strategy, or Facade.

For example, in Spring:

Logging

Security

Transactions

Metrics

may sometimes be better implemented using:

AOP

Filters

Interceptors

Spring Security

@Transactional

Micrometer

We should choose the simplest suitable design.

Class 1 Pattern Structure

We should remember these four participants:

Participant	Our Example
Component	NotificationSender
Concrete Component	EmailNotificationSender
Decorator	NotificationDecorator
Concrete Decorator	LoggingNotificationDecorator
Concrete Decorator	EncryptionNotificationDecorator

Concrete Decorator	MetricsNotificationDecorator
--------------------	------------------------------

Complete Demo

```
public class DecoratorPatternDemo {

    public static void main(String[] args) {

        NotificationSender sender =
            new MetricsNotificationDecorator(
                new LoggingNotificationDecorator(
                    new EncryptionNotificationDecorator(
                        new EmailNotificationSender()
                    )
                )
            );

        sender.send(
            "Your booking is confirmed."
        );
    }
}
```

Object structure:

```
Metrics
 |
Logging
 |
Encryption
 |
Email
```

Client still sees:

NotificationSender

and calls:

```
sender.send(message);
```

That is the beauty of Decorator Pattern.

Homework — Storage Decorator

Suppose we have:

```
public interface StorageService {  
  
    void upload(  
        String filename,  
        byte[] content  
    );  
  
    byte[] download(  
        String filename  
    );  
}
```

Basic implementation:

```
public class LocalStorageService  
    implements StorageService {  
}
```

Create these decorators:

LoggingStorageDecorator

EncryptionStorageDecorator

CompressionStorageDecorator

MetricsStorageDecorator

Students should be able to compose:

```
StorageService storage =  
    new MetricsStorageDecorator(  
        new LoggingStorageDecorator(  
            new EncryptionStorageDecorator(  
                new CompressionStorageDecorator(  
                    new LocalStorageService()  
                )  
            )  
        )  
    );
```

Homework Requirements

1. Create StorageService.
2. Create LocalStorageService.
3. Create abstract StorageDecorator.
4. Create LoggingStorageDecorator.
5. Create EncryptionStorageDecorator.
6. Create CompressionStorageDecorator.
7. Create MetricsStorageDecorator.
8. Upload and download a file successfully.
9. Change decorator order and explain the effect.
10. Explain why inheritance would become difficult.
11. Identify the SOLID principles demonstrated.
12. Add an S3StorageAdapter and decorate it using the same decorators.

Interview Questions for Class 1

1. What problem does Decorator Pattern solve?
 2. Why use Decorator instead of inheritance?
 3. What are the main participants in Decorator Pattern?
 4. Why must a Decorator implement the same interface as the wrapped object?
 5. What does the Decorator store internally?
 6. Can multiple decorators wrap the same object?
 7. Does decorator order matter?
 8. How does Decorator support Open/Closed Principle?
 9. What is the difference between a wrapper and a Decorator?
 10. Can Decorator and Adapter be used together?
 11. What is the main disadvantage of Decorator?
 12. When would you choose framework AOP instead of Decorator?
- We'll answer these in Class 2 after exploring the pattern more deeply.